

P?S/8 BINCON Technical Guide [preliminary]

Copyright © 1975-1989 Charles Lasner Associates

Copyright © 1990-2016 »CLA Systems

CLASystems@gmail.com

Last edit: 03-Jul-2016 cjl

Note: This document generally follows the conventions of the P?S/8 Keyboard Monitor Command Guide available separately.

This is a preliminary specification for a new P?S/8 utility to be known as *BINCON*.

Note: BINCON is being introduced as an add-on to existing copies of P?S/8; as such, it may only be present on certain system release media. Appendix A contains information on using BINCON on P?S/8 Version 8Z systems should BINCON not be present in the System Programs Directory.

The BINCON utility is designed to solve several specific problems caused by design limitations of OS/8 PAL8. When used to develop certain large PDP-8 programs [such as the source files of the P?S/8 operating system], it is not possible to use the DTORG directive. This is due to compromises made to PAL8 early on in the development of OS/8 to comply with short-sighted demands made by the Typeset-8 group.

By implementing a utility to post-process the binary output of OS/8 PAL8, P?S/8 BINCON allows source code to be developed from either operating system allowing free usage of the DTORG directive.

Additionally, the complete feature set of P?S/8 PAL can be used to develop binary programs capable of being run in the OS/8 environment. PAL language enhancements include support for conditional literals and dual-mode LINC/PDP-8 assembly targeted for the LINC, LINC-8 or PDP-12.

Note: With regard to LINC assembly implementation, P?S/8 PAL is compatible with the PDP-12 LAP6-DIAL/DIAL-MS assembler including the use of all PDP-8 literal types. The DIAL assembler does not support literals; P?S/8 PAL allows free usage of literals to construct any appropriate PDP-8 coding. For more information on P?S/8 PAL features, the reader is directed to the P?S/8 PAL Version Technical Guide available as a separate document.

Prior to the availability of P?S/8 BINCON, only the original historic method of generating P?S/8 from the source files could be implemented. This involves using an internal system generation routine to write out various components loaded into specific memory locations to the designated target storage blocks.

This was implemented such that 12K to 16K is required [depending on whether the P?S/8 system device handler for the target system requires an extended-memory section]. In some cases, this requirement exceeds available memory on the intended target system; various roundabout mechanisms had to be created to indirectly bring a working operating system to the intended target hardware.

Future system generation and update mechanisms will depend on the use of the DTORG directive to allow implementation of a standalone storage block-writing utility program. In nearly all cases, such a utility can be run in 4K ensuring that all systems can be created in available memory.

Note: The historic method will continue to be supported primarily for the benefit of P?S/8 developers. In certain instances, conditional assembly techniques may be necessary to allow PAL8 to assemble the source code [unless P?S/8 BINCON is available to post-process the binary output of PAL8].

When the DTORG directive is used in PAL8 [and P?S/8 BINCON is not available to post-process the binary output], it is impossible to load the PAL8 binary output file with OS/8 *ABSLDR*. This is by design as part of the Typeset-8 compromises. Punching the OS/8 binary files to binary paper-tapes creates output that can only be read with a special Typeset-8 block-writing utility; the binary paper-tapes are not compatible with the standard BIN loader.

BINCON implements bi-directional binary file conversion between P?S/8 PAL binary output and OS/8 PAL8 binary output. When BINCON encounters a DTORG frame in the PAL8 binary file, P?S/8 binary output will contain corrected DTORG output consistent with the new definition of DTORG as described in this document. As a result, the P?S/8 binary files can be loaded [or even punched to binary paper-tape in BIN or RIM format using standard P?S/8 utilities]. Conversion of the resultant P?S/8 binary files back to OS/8 binary format yields a compatible file that can be loaded by OS/8 *ABSLDR*. Note: Certain additional features are available in BINCON oriented towards peculiarities of the P?S/8 filesystems and are included for completeness and symmetry.

Appendix B describes the derivation of the replacement implementation for the DTORG directive as defined by the author of this document. A compatible implementation will be made available as part of the next upgrade to P?S/8 PAL. Note: While this implementation has been slated for implementation, without a satisfactory method of implementing the corrected assembler feature in OS/8, no other PDP-8 development project could ever take advantage of this feature of the PAL language. The actual P?S/8 PAL implementation involves a small change at the source code level [likely involving about 10-12 lines of PAL source code].

Individuals interested in revamping OS/8 PAL8 DTORG support can contact the author at the e-mail address indicated above. Unfortunately, the nature of distribution of P?S/8 cannot depend on such good work even if implemented widely due to overall P?S/8 development principles of distribution to the widest range of potential users.

1. BINCON design principles.

The primary reason for the creation of the BINCON utility is to expand the P?S/8 development regimen, which demands that OS/8 can create a P?S/8 starter system from the P?S/8 source or binary files. Available memory must be 12K [or 16K if the target P?S/8 system device handler requires extended memory].

By design, development of P?S/8 source programs must be generally compatible with the specific limitations of OS/8 PAL8. No P?S/8 superset features are used; this aids in the general distribution of P?S/8 to specific user sites that may not have direct access to P?S/8. Supporting the proposed distribution method described below will allow an alternative to the current method [which will be retained for the benefit of P?S/8 developers].

Note: While a functional P?S/8 system is also capable of creating the P?S/8 starter system, often P?S/8 is not already available. The alternative method will allow systems capable of running the RIM and BIN loaders the opportunity to create a starter [or complete] P?S/8 system for any target system. However, the DTORG directive must be functional to create such an installation mechanism.

BINCON allows the DTORG directive to be useful to development of PDP-8 systems, especially the alternative system generation method for P?S/8 as described in this document. OS/8 systems cannot use this standard feature of PAL8 due to mis-implementation of DTORG to accommodate the short-sighted needs of the Typeset-8 group; the alternative described in this document allows all users equitable access to this feature.

When DTORG directive usage is well-integrated into the relevant P?S/8 source code files, the alternative method of system generation will be made available to a wider user audience. This will allow implementing a smaller program that runs as a standalone block-writing utility on any system of at least 4K memory. [More memory will be needed only if the P?S/8 system device handler requires extended memory.]

Distribution files will be released that are compatible with the standard BIN loader; the block-writing utility will be loaded and used to write out the P?S/8 system components on the other binary paper-tapes. Note: The block-writing utility will also run as a P?S/8 system program with access to P?S/8 binary files; this will avoid the need for paper-tape input.

2. BINCON features.

P?S/8 BINCON will convert P?S/8 slurp binary files to OS/8 binary files [for use with OS/8 *ABSLDR*] as well as the conversion in the opposite direction.

BINCON identifies the incompatible frames within the OS/8 binary output and replace them [in the P?S/8 output data] with the compatible equivalent as described above. [If feasible, the input data will be checksummed perhaps by using techniques involving multiple passes over the data.]

By using P?S/8 BINCON, binary files can be loaded in P?S/8 despite the presence of DTORG statements created in the OS/8 environment.

Additionally, OS/8 format binary files can be written out to OS/8-compatible media that can load in the OS/8 environment despite the presence of DTORG statements, even if the assembly originated with OS/8 PAL8.

If the '/E' command-line option switch is set, P?S/8 binary output will be written to extended-length files. This will typically allow a single P?S/8 binary output file to contain the entire OS/8 binary file contents. [Note: P?S/8 *BIN* is capable of loading extended-length binary files in *Slurp* format; each block of a Slurp format binary file is independent of all other blocks in the file other than the prevailing loading field and origin from the previous block; every file starts with origin and field settings. The final block indicates logical end of file; there are no inherent file length considerations.]

By using BINCON to convert several P?S/8 TFS binary files to OS/8 format, then using BINCON to convert the OS/8 binary file to an extended-length file, only one extended-length file need be specified to load the equivalent of several TFS binary files. This represents not only a potential convenience, truly large projects could benefit by having each module of a project create a single output file for ultimate composite loading.

Other command-line option switches may be implemented in BINCON as development of the program progresses.

Appendix A.

Using BINCON on P?S/8 systems without BINCON installed.

BINCON is a recent addition to releases of P?S/8 Version 8Z; it will be included in all future releases as a standard system program compatible with the concise or extended forms of program execution.

Those familiar with the P?S/8 system programs directory can add BINCON as long as there is sufficient free storage space. Failing this, it is also possible to run BINCON as an impromptu system program as described below; the binary file[s] created by assembling BINCON source file[s] must be available on an available logical device unit [drive 1 in this example].

Console output	Comments
.	Keyboard monitor prompt.
.CA:1	CA{talog} command for drive 1.
BINC�1 BINC�2 README RDMETU	Contents of TFS directory on drive containing binary files.
.LC:0	Avoid potential system dependencies on data drive.
.GET BINC�1:1 BINC�2	Create program image.
.	System restarts here.
.R \$ [Files and other parameters]	Particulars as used when BINCON is already installed.

The R [or RU{n}] command specifying the program name \$ [or %] executes the default core image as setup by the latest GET operation or equivalent. [Note: This will function permanently unless some other system usage or command changes the contents of the % and \$ files. If necessary, the GET operation can be repeated to refresh the default program image.]

BINCON is designed to be compatible with the work-around mechanism described above to allow rapid addition to existing systems; this mechanism was introduced in P?S/8 Version 8Y and is supported on all newer releases.

Usage restrictions.

There are no usage restriction associated with BINCON. The program does not use the =xxxx construction; there is no internal overlay structure within the program. The program is designed to start at 00200 to be compatible with the default program execution mechanism.

End of Appendix A.

Appendix B.

History of the implementation of the DTORG directive and a new compatible replacement.

1. Early implementation of DTORG for the DEC Typeset-8 group.

On systems based on logical blocks of 128 words per block [such as The R-L Monitor System, POLY BASIC and related derivative operating systems, P?S/8 and Typeset-8] it is useful to designate specific logical blocks to write out particular operating system components.

For example, the P?S/8 system device handler and integrated boot-up block must be written to logical block 0000 of the target operating system being created.

Presently, this is accomplished by a large system generation utility that requires 12K-16K [depending on whether the system device handler requires an extended-memory section] as part of a complete routine to create a bootable starter system. This memory requirement may exceed the available memory on the intended target system; elaborate workarounds must be contrived presently to create these systems.

If each target block can be designated as part of a stand-alone block installation program, it is likely that all systems can be created on the actual intended hardware regardless of memory size. [The only requirement would be that a suitable non-system device handler was available that can fit into the available memory size; this would usually be the case even if the target machine was 4K.]

Typeset-8 takes advantage of the DTORG directive to implement this method of system generation on 4K Typeset-8 systems. However, due to specific implementation limitations [that are arguably needless] the prevailing assemblers cannot be used to benefit development of any other operating system.

The reason for this is that Typeset-8 is created from binary paper-tapes that are deliberately incompatible with all binary standards including the RIM and BIN loaders.

The BIN file format does not support the DTORG directive output as implemented for Typeset-8. An unsupported frame type is used to start the data intended to be written out on the block described by the DTORG frame pair. [Note: The end of the data to be written is defined as all data prior to an origin setting outside of the current page as defined within the assembly; this part of the design is rational and quite acceptable.]

Due to the presence of the unsupportable frame, it is impossible for the binary paper-tape to pass a checksum test as defined in the BIN loader specification. As such, only the Typeset-8 program could confirm the validity of the data due to the incompatible format.

Note: The previous sentence above is pure speculation; it may not be possible to confirm the validity or at the least it would require the paper-tape be read more than once, which is onerous. Clearly, they never investigated this notion and merely ignored validation.

The Typeset-8 block-writing utility program doesn't confirm validity of the paper-tape, merely writing out each data block as encountered.

Note: Many Typeset-8 systems were created using the low-speed reader on Teletype Model 33 ASR or 35 ASR terminals; these devices are known for reliability problems unless maintained on a schedule that is beyond impractical. Undoubtedly, Typeset-8 development suffered some setbacks due to this rather poor design, occasionally writing out incorrect and unintended data.

When Typeset-8 development meant much to DEC management, the Typeset-8 group was able to convince the developers of TOPS-10 PAL10 to support the distorted binary output format. Very long paper-tapes were created and hand-walked to Typeset-8 machines to be read into memory [perhaps in some cases for hours at a time] as part of their rather disorganized attempt at software development. Since TOPS-10 has no built-in requirement for compatibility with the PDP-8 BIN loader, this mis-feature was easily added to PAL10. Unfortunately, expediency was the dominant factor in adding DTORG to PAL10, not the needs of the general user audience for unrelated program development.

Note: It appears that at some point, DEC management wasn't even aware there were PDP-8 programming experts available outside of the meager number of people working on in-house development systems at the time. When it became clear many "outsiders" existed, some being hired by DEC, no one ever revisited this feature and became essentially moribund. Part of the problem is there is no direct benefit to OS/8 development to use the DTORG feature. Yet the methods described here using DTORG also apply to other DEC products. [About the only products the concept doesn't apply to are the Disk Monitor System and OS/8.]

However, this issue had come up even before the OS/8 project was contemplated:

POLY BASIC was developed [well over 95%] at Brooklyn Poly on the R-L Monitor System [the forerunner of P/S/8] on a configuration consisting of a 4K PDP-8 CPU, a TC01 and a single TU55 DEctape drive. At the time, POLY BASIC only supported this configuration.

The authors of POLY BASIC [Richard Lary and Lenny Elekman] became DEC employees and brought POLY BASIC with them to demonstrate to DEC management, who were very impressed with this standalone BASIC system. They were "encouraged" to add on support for all then-current other storage devices; development was moved to PAL10 on an in-house TOPS-10 system.

While the conceptual problem of creating POLY BASIC from a few binary modules parallels the same development issues for P?S/8 as described in this document, the authors were forced to reject the suggested method because the Typeset-8 group had influenced the maintainers of PAL10 to implement DTORG in an incompatible manner; binary paper-tapes produced by PAL10 cannot be loaded with the standard BIN loader.

As this was unacceptable, they implemented a slightly more "guided" approach to the same overall method of system generation, even though it was obvious the alternative method described here would be advantageous. The net result is a very long binary paper-tape set that used the BIN loader for all operations, but still requires a central system generation program; the modules cannot target the intended storage blocks.

At some time later, DEC decided to co-opt POLY BASIC and revamp it slightly into EDU-System 30 [and later EDU-System 15/30 for the TD8E only]. Because of the additional overhead of a hamstrung development method, DEC assigned fledgling programmers to finish what should have been a simple modification; in the end Richard Lary had to help the new programmers bring the product[s] to functioning condition.

DEC management clearly had an opportunity to make decisions about how to modify the prevailing assembler for use with more active products than the stagnant Typeset-8. Unfortunately, when OS/8 was being developed, no changes were made; OS/8 PAL8 just implemented DTORG identically to PAL10 to the benefit of no one.

When OS/8 [or actually one of its predecessors] were in the early design stages, the binary file format was deliberately made far more inefficient than necessary merely to slavishly continue this poor design mechanism.

It is clear that the far more efficient *Slurp* format was known to all parties involved at the time; however Slurp format was rejected in favor of placating the Typeset-8 group's demands without ever thinking through how all parties could be accommodated without needless overhead that effectively locks out program development of another block-writing utility unless completely compatible with the limitations of Typeset-8.

Very early in the overall design period of OS/8, the author of this document suggested a practical alternative implementation of DTORG. However, it was never considered and he was never a part of the design team. [Note: He was asked to join the development group, but rejected DEC's offer of employment. Instead, he chose to develop the R-L Monitor System along with some colleagues; this lead to the current implementation of P?S/8. It is noteworthy that at least four other reknowned PDP-8 experts, all employees of DEC at the time, also rejected the project that would eventually be known as OS/8.]

Note: Several other programs have been written using analogous design principles, most notably OS/8 *BUILD*. BUILD imparts special information within a compatible binary file using binary constructs that are vaguely analogous to the suggested mechanism to implement DTORG.

2. Compatible implementation of DTORG directive support.

By clever design, it is possible to maintain the integrity of the binary file format for all purposes whether the DTORG directive is involved or not.

Note: With the burden of incompatible frames lifted, it would have been possible to implement Slurp format in OS/8; had this been accomplished, all binary loading operations for all users over the years could have been carried out in far less time. Slurp format was demonstrated by the author to various DEC employees; binary file loading time using Slurp format is profoundly faster than any file format derived from paper-tape frames such as OS/8's .BN files; a DECTape-based P?S/8 system actually loaded large sample binary files **faster** than an RK05-based OS/8 system despite the large difference in basic data transfer rate of the two storage devices.

A few of the design principles of BUILD were adapted to create compatible DTORG output yet maintain overall binary compatibility with all known loading programs.

This is accomplished by the use of special origin settings in the file without any corresponding data word loading. The sequence of origin settings is meaningless to the standard binary loaders [which handle the binary file correctly as long as the standard BIN checksum is correct within the file]. However, a special program aware of the presence of the origin groups can implement the intent of the DTORG directive.

Note: If the Typeset-8 group remained unmovable and insisted their demands be met, this could still be done in a way that only affected their interests without harming that of all other users. A conversion program [perhaps another option to OS/8 *PIP*] could convert compatible binary files to Typeset-8 compliant binary paper-tapes]. Had that been done, P?S/8 BINCON would be largely unnecessary.

The following PAL coding is the equivalent of what the DTORG directive does [to be released after the next major edit of P?S/8 PAL]:

```
/      DTORG      DTBLOCK      /SOURCE CODE PROCESSED BELOW.

      DTBLOCK=XXXX      /BLOCK EVALUATED FROM DTORG STATEMENT.

      FOO=      .      /WE NEED THE CURRENT ORIGIN SETTING.

      *7777      /START OF DTORG SECTION. NEVER USED IN
                  /PRACTICAL PDP-8 PROGRAMMING OUTSIDE
                  /OF PERHAPS THE BIN AND RIM LOADER.
      *DTBLOCK      /DTORG VALUE FOR SPECIAL LOADER.

      *FOO      /RESTORE THE PREVIOUS ORIGIN SETTING.
```

Internally, P?S/8 PAL has access to all of this information; an internal routine returns the binary value of the passed block argument. If the argument is not present, the returned value is 0. The corresponding origin settings are added to the binary output file stream; for conventional loading purposes, there is no net change in the binary output. As such, the historic method of development can function as intended.

Only a special block-writing utility aware of the special meaning of the origin settings can make use of this added data. Implementation of the corrected DTORG directive in P?S/8 PAL will require less than 10 additional words of PDP-8 code over the current incomplete routine. [Currently, P?S/8 PAL acknowledges the DTORG directive by processing the argument, then otherwise ignoring the statement.]

Note: The specification for a functional block-writing utility is outside of the scope of this document. Once meaningful use of the DTORG directive is added to the various P?S/8 source files, a formal specification will be drawn up leading to implementation of the alternate system generation block-writing utility.

End of Appendix B.

[End-of-file]